

System Architecture Overview

eloomi

Table of Contents

Introduction	1
Architecture	2
Front-end hosting	3
Compute Resources	5
Databases	6
Multitenancy	8
Data encryption	9
Data in transit	9
Data at rest	10
Application secrets management	11
CICD process	11
Monitoring and logging	11

Introduction

At eloomi, we are dedicated to building a platform that empowers organizations with reliable and transformative solutions. We prioritize the creation of a secure and dependable ecosystem, instilling trust in our clients. From the foundation of our system architecture to the extensive network of our global infrastructure, networks, and information systems, eloomi prioritizes availability, security, and confidentiality.

Our talented and qualified team is a cornerstone of our platform's strength. Our employees form the essential backbone of shaping and upholding our resilient and robust system architecture. Through accurate Human Resources processes, eloomi always ensures that only the most skilled and qualified professionals contribute to managing and overseeing the core infrastructure supporting our solutions.

Fueled by advanced information systems and technologies, our architecture excels in operational performance while safeguarding against risks and threats. This steadfast commitment to protection aligns seamlessly with our promise to customers — ensuring the availability, security, and confidentiality of their data and operations.

This document serves as a comprehensive guide, offering our customers insights into our system architecture. By understanding the foundations of our platform, clients can gain a clear view and a deeper appreciation for how eloomi exceeds the expectations set for availability, security, and confidentiality commitments on a global scale.



Architecture

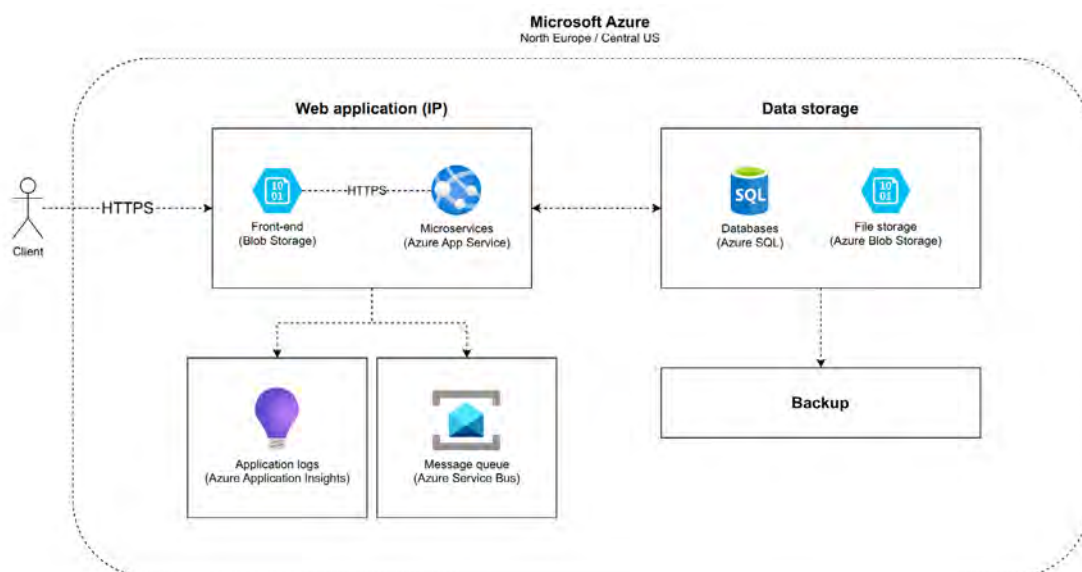
eloomi's platform infrastructure is hosted on Microsoft Azure. Our resources are located in Azure Datacenters in Northern Europe(Ireland) and Central US (Iowa) for two separate environments for Europe and the USA. All Customers are using common SaaS infrastructure, but with implemented multi-tenancy options (i.e data of each customer is segregated in its own database schema) so data is securely stored and managed in compliance with industry standards.

Our Continuous Development strategy involves using staging and production environments for both regions. Staging mirrors production for infrastructure components and traffic flow.

eloomi's application is designed using a microservice architecture, currently consisting of approximately 15 services.

All services are utilizing a similar tech stack:

- Compute Resources: App Service for hosting .NET
- C# for the solution
- Application Insights for logging
- SQL database for data storage
- Key vault for configuration and secret management
- Blob storage for file storage
- Service bus for asynchronous event communication
- Azure Monitor for performance metrics and alerts

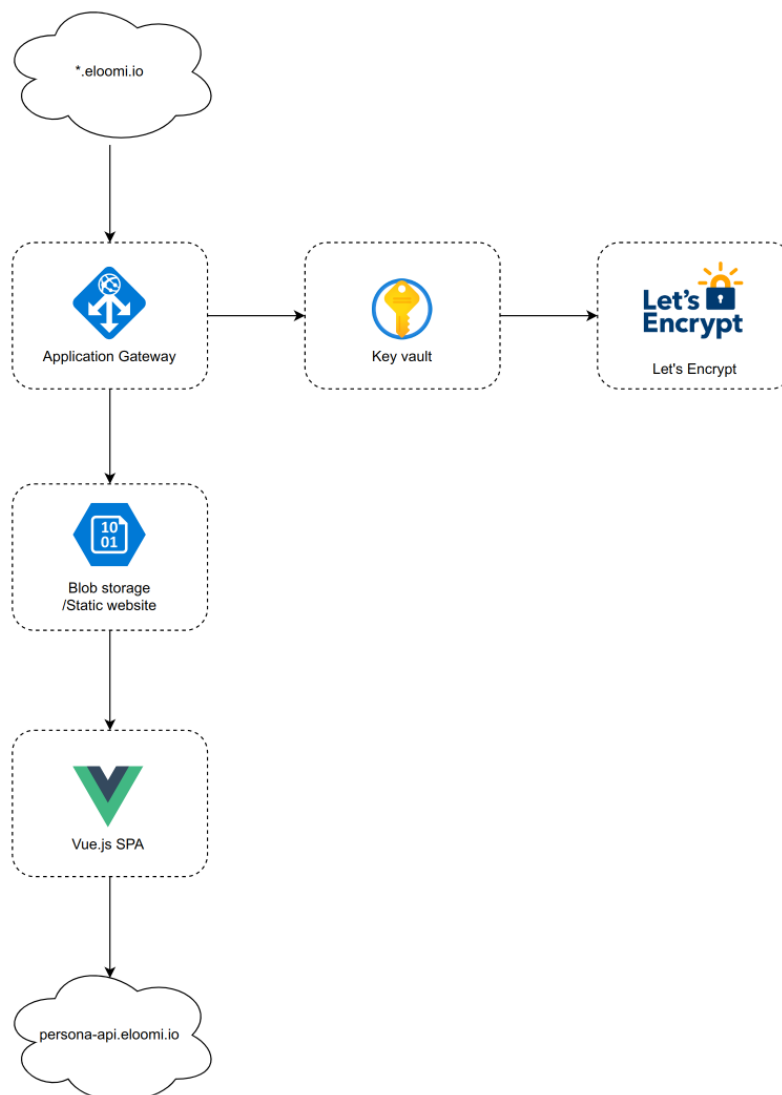


Front-end hosting

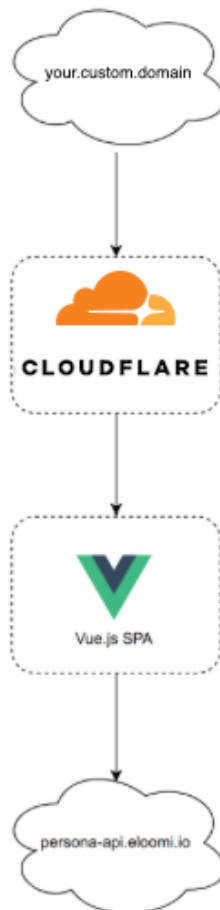
The front-end is a Vue.js SPA application that is stored on Blob Storage, using the Static website functionality. Clients request assets from the front-end through a *.eloomi.io domain (e.g. tech-team.eloomi.io).

All requests are handled by an Application Gateway component, which encrypts all traffic with a Let's EncryptSSL certificate.

The client communicates with the back-end via the persona-api.eloomi.io domain, which is the API for our front-end.



For customers who are using custom domains, the entire SPA is hosted on Cloudflare Pages, with domain and SSL certificate administration being overseen by Cloudflare. In that case, a CNAME record should be created and pointed to domains.eloomi.io.



Compute Resources

Each eloomi service is hosted on a separate Azure AppService, utilizing either a common or dedicated Azure AppService Plan. By hosting our app this way, we benefit from Azure-provided security, load balancing, broad scaling options (vertical and horizontal) and automated management for each of our microservices. Azure's high availability solutions ensure that Azure App Service will be available 99.95% of the time.

Underneath the surface, App Service utilizes a load-balancing layer and pre-created web workers to ensure seamless high availability mechanisms and live scaling. When the App Service Plan triggers a scale-out operation (such as adding two more workers), the available workers are allocated from a pre-existing pool of ready-to-go workers. These workers are already pre-provisioned and warm, so our service simply needs to get deployed on the worker. Once that happens, the worker is added to the rotation and the front end can begin allocating traffic to it. This entire process is typically completed in just a few seconds.

In addition to the scaling options provided by Azure (scaling up and scaling out) for services that require a lot of resources, we are dividing a service into two parts - one responsible for handling user/api traffic and the other for processing backend jobs. We can also scale both parts independently making the process even more granular.

The platform components of App Service, storage, network connections, web frameworks, management and integration features, are actively secured and hardened by Microsoft. App Service goes through vigorous compliance checks on a continuous basis to make sure that:

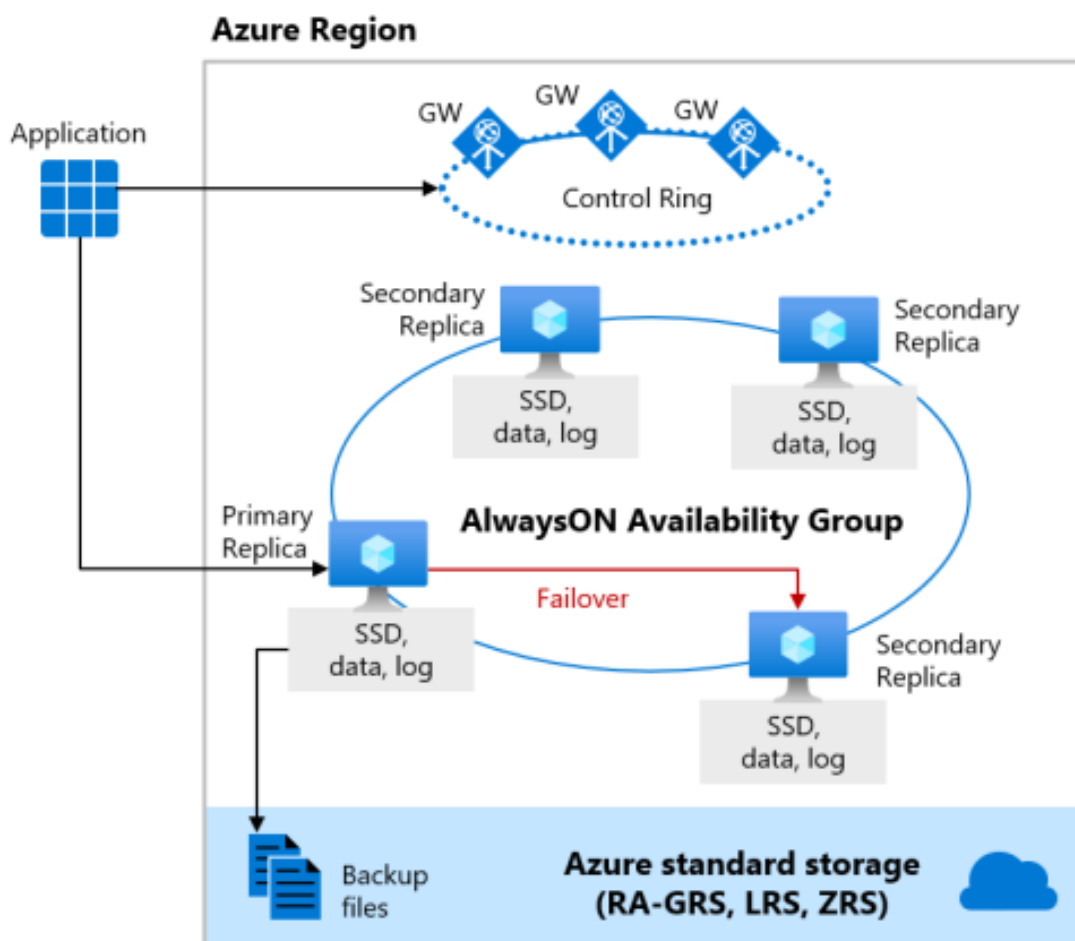
- eloomi's App resources are secured and separated from resources of other customers of Azure.
- Communication of secrets (such as connection strings) between app and other Azure resources (such as SQL Database) stays within Azure and doesn't cross network boundaries. Secrets are always encrypted when stored.
- All communication over App Service connectivity features are encrypted.
- Connections with remote management tools like Azure PowerShell, Azure CLI, Azure SDKs, REST APIs, are encrypted.
- 24-hour threat management protects the infrastructure and platform against malware, distributed denial-of-service (DDoS), man-in-the-middle (MITM), and other threats.

Databases

All data is stored in SQL server, which currently utilizes one shared SQL pool (elastic pool)for managing all databases. Each service has its own SQL database. Each tenant (internally referred to as a platform) has its own schema inside the database.

Customers have different and unpredictable usage patterns, making it hard to anticipate their individual database resource requirements. To address this, we use elastic pools as a database layer. These pools ensure that databases receive the performance resources they require when they need them. Additionally, they offer a straightforward resource allocation mechanism within a predictable budget.

Azure SQL Database service automatically ensures all the databases are online and healthy, and constantly strives to achieve the published SLA levels.



High availability is implemented using a technology similar to SQL Server Always On availability groups. The cluster includes one primary replica that can be accessed for read-write customer workloads and up to three secondary replicas for computing and storage containing data copies. If a failover occurs, Azure SQL connections are automatically redirected to the new primary replica or readable secondary replica.

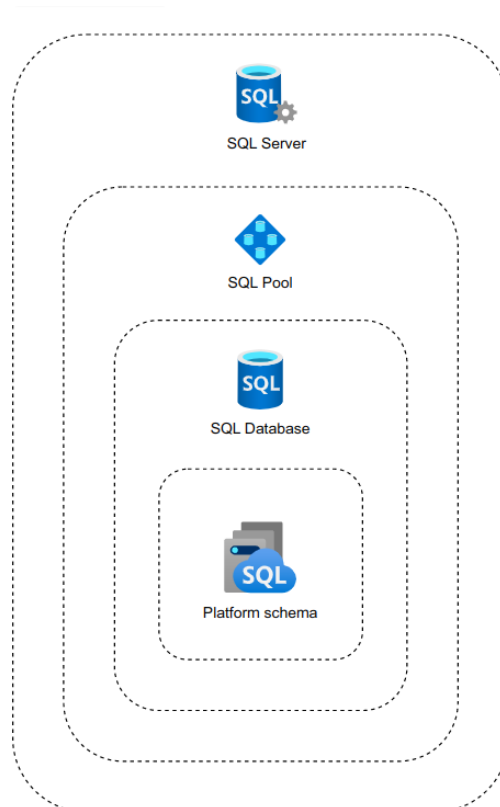
For all deployed databases eloomi is using Geo-redundant storage that is duplicating data in another region within the same data residency region (data hosting and storage will still be performed within the selected region), protecting from primary region outages.



Enabled automated backups for SQL with point-in-time and long-term restoration options increasing data safety level. From the maintenance point of view, Terraform deployment monitors manual changes and configuration drifts on SQL resources, while “Azure resource locks” prevent accidental resource deletion. Additionally, firewall access configuration restricts access to internal eloomi networks.

Multi tenancy

Multi-tenancy is implemented on the API gateway, service and database layer. All data is stored in SQL server, which currently is utilizing one shared SQL pool for managing all databases. Each service has its own SQL database, and each tenant (internally referred to as a platform) has its schema inside the database.



Data encryption

To ensure proper and effective use of cryptography to protect the confidentiality, authenticity and/or integrity of information, eloomi has defined a policy on the use of cryptographic controls for the protection of information.

Data in transit

Data in transit for *.eloomi.io domains is encrypted with TLS 1.2 and cypher suites with 128-bit or better encryption.

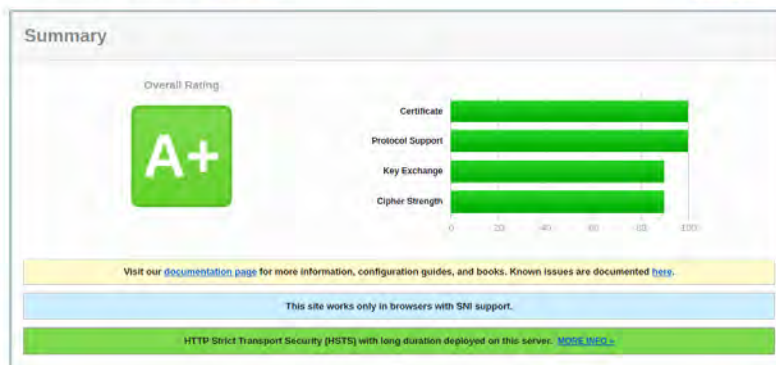
SSL certificates are issued through LetsEncrypt and are renewed every two months by an automated process.

The eloomi domains show an overall score of “A+” on Qualys SSL report on the platforms routed via AppGateway and a score of “A” for platforms routed through Cloudflare.

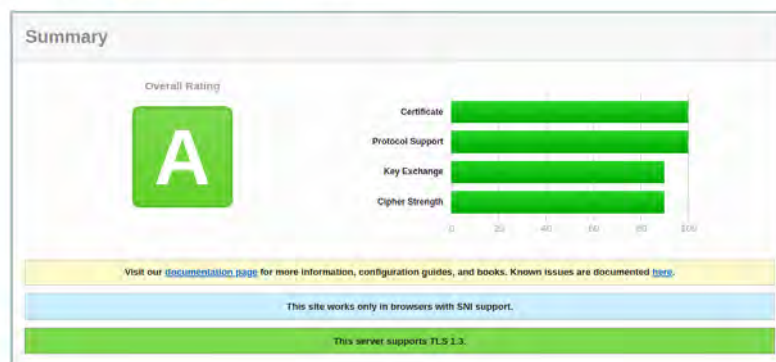
SSL Report: team.eloomi.io (20.82.210.126)

Assessed on: Mon, 11 Sep 2022 11:51:17 UTC | [Hide](#) | [Clear cache](#)

[Scan Another »](#)



SSL Report: domains.eloomi.io (172.66.47.184)



For service to service communication through eloomi's API we are using Microsoft signed certificates in *.azurewebsites.net domain also with an "A" grade :



Communication between services and SQL databases is also always encrypted by forcing the following parameters in all connection strings:
Encrypt=True;TrustServerCertificate=False;

Data at rest

Data at rest is encrypted and decrypted transparently using 256-bit AES encryption, one of the strongest block cyphers available, and is FIPS 140-2 compliant. Data in storage accounts is encrypted using Microsoft-managed keys: Azure Storage encryption for data at rest. Data encryption is enabled on all data storage accounts, databases and service bus instances.

All eloomi's SQL databases have Transparent data encryption (TDE) feature enabled, which helps protect Azure SQL Database against the threat of malicious offline activity by encrypting data at rest. It performs real-time encryption and decryption of the database, associated backups, and transaction log files at rest without requiring changes to the application.

TDE performs real-time I/O encryption and decryption of the data at the page level. Each page is decrypted when it's read into memory and then encrypted before being written to disk. TDE encrypts the storage of an entire database by using a symmetric key called the Database Encryption Key (DEK). On database startup, the encrypted DEK is decrypted and then used for decryption and re-encryption of the database files in the SQL Server database engine process.

Application secrets management

Service configuration parameters and sensitive information, such as application passwords and tokens, are securely stored in Azure Key Vault. The eloomi services code includes references to the key vault data.

Our key vaults are “Standard” tier without a hardware security module (HSM) enabled. According to Microsoft best practices we have separate key vault instances per environment with purge protection turned on. Access to regional key vaults is secured by Microsoft RBAC rule based on security groups.

CI/CD process

eloomi is using Bitbucket pipelines in order to test and deploy both application and infrastructure changes. For application deployment, we are using another Azure AppService feature which is Deployment Slot (Blue/Green deployment) - using this approach we are able to deploy new versions without interrupting user experience. Common modules in eloomi app are securely stored as Azure DevOps artefacts and referenced during the build process.

All Infrastructure components and their state are managed by Terraform steps in pipelines with state files securely stored on blob storage containers.

To make change verification easier and faster, and improve code quality and security eloomi relies on external tools i.e. sonarcloud, snyk.

Monitoring and logging

All components in eloomi architecture are continuously monitored for various aspects, such as resource performance metrics (CPU, memory, and storage consumption) a number of triggered jobs or events, and appinsights logs. Monitoring is based on Azure monitor data for different resources and prometheus data gathered by various metric exporters.

Cyclic scalability and load tests help to predict how infrastructure components should grow with the number of customers.

Whenever an anomaly is detected on critical metrics, the Infrastructure team receives notifications of issues through various alerts configured on the monitoring stack.

Trusted by hundreds of companies worldwide

Let's chat

